REST API Basic Interview Questions

1. What do you understand by RESTful Web Services?

RESTful web services are services that follow REST architecture. REST stands for Representational State Transfer and uses HTTP protocol (web protocol) for implementation. These services are lightweight, provide maintainability, scalability, support communication among multiple applications that are developed using different programming languages. They provide means of accessing resources present at server required for the client via the web browser by means of request headers, request body, response body, status codes, etc.

Rad pages 33 for details

2. What is a REST Resource?

Every content in the REST architecture is considered a resource. The resource is analogous to the object in the object-oriented programming world. They can either be represented as text files, HTML pages, images, or any other dynamic data.

- The REST Server provides access to these resources whereas the REST client consumes (accesses and modifies) these resources. Every resource is identified globally by means of a URI.

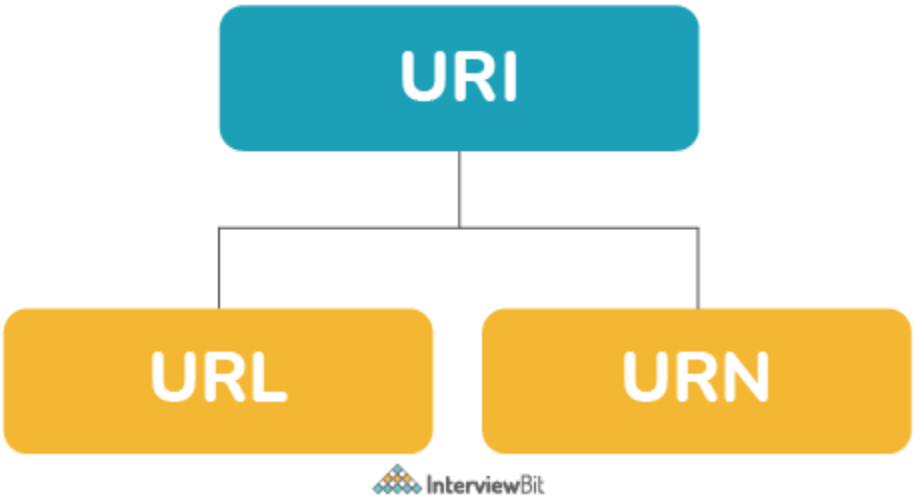
3. What is URI?

Uniform Resource Identifier is the full form of URI which is used for identifying each resource of the REST architecture. URI is of the format:

```
<protocol>://<service-name>/<ResourceType>/<ResourceID>
```

There are 2 types of URI:

© Copyright by Interviewbit



© Copyright by Interviewbit

- **URN:** Uniform Resource Name identifies the resource by means of a name that is both unique and persistent.
  - URN doesn't always specify where to locate the resource on the internet. They are used as templates that are used by other parsers to identify the resource.
  - These follow the `urn` scheme and usually prefixed with `urn:` . Examples include
    - `urn:isbn:1234567890` is used for identification of book based on the ISBN number in a library application.
    - `urn:mpeg:mpeg7:schema:2001` is the default namespace rules for metadata of MPEG-7 video.
  - Whenever a URN identifies a document, they are easily translated into a URL by using "resolver" after which the document can be downloaded.
- **URL:** Uniform Resource Locator has the information regarding fetching of a resource from its location.
  - Examples include:
    - `http://abc.com/samplePage.html`
    - `ftp://sampleServer.com/sampleFile.zip`
    - `file:///home/interviewbit/sampleFile.txt`
  - URLs start with a protocol (like ftp, http etc) and they have the information of the network hostname (sampleServer.com) and the path to the document(/samplePage.html). It can also have query parameters.

© Copyright by Interviewbit



4. What are the features of RESTful Web Services?

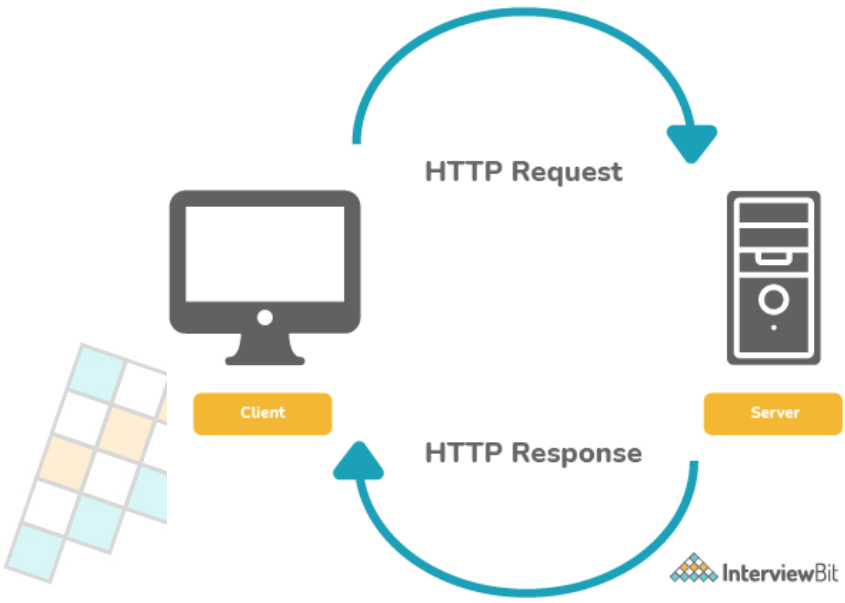
Every RESTful web service has the following features:

- The service is based on the Client-Server model.
- The service uses HTTP Protocol for fetching data/resources, query execution, or any other functions.
- The medium of communication between the client and server is called "Messaging".
- Resources are accessible to the service by means of URIs.
- It follows the statelessness concept where the client request and response are not dependent on others and thereby provides total assurance of getting the required data.
- These services also use the concept of caching to minimize the server calls for the same type of repeated requests.
- These services can also use SOAP services as implementation protocol to REST architectural pattern.

5. What is the concept of statelessness in REST?

© Copyright by Interviewbit

The REST architecture is designed in such a way that the client state is not maintained on the server. This is known as statelessness. The context is provided by the client to the server using which the server processes the client's request. The session on the server is identified by the session identifier sent by the client.



State typically includes things like:
.Authentication status
.User preferences
.Previously entered form data
.Shopping cart contents
.Any information about past interactions

6. What do you understand by JAX-RS?

As the name itself stands (JAX-RS= Java API for RESTful Web Services) is a Java-based specification defined by JEE for the implementation of RESTful services. The JAX-RS library makes usage of annotations from Java 5 onwards to simplify the process of web services development. The latest version is 3.0 which was released in June 2020. This specification also provides necessary support to create REST clients.

In this context, "specification" refers to a standardized set of rules, interfaces, and guidelines defined by Java EE (Jakarta EE) that outlines how RESTful web services should be implemented in Java.

7. What are HTTP Status codes?

These are the standard codes that refer to the predefined status of the task at the server. Following are the status codes formats available:

© Copyright by Interviewbit

- 1xx - represents informational responses
- 2xx - represents successful responses
- 3xx - represents redirects
- 4xx - represents client errors
- 5xx - represents server errors

Authentication error 401 Unauthorized
Authorization error 403 Forbidden

Most commonly used status codes are:

- 200 - success/OK
- 201 - CREATED - used in POST or PUT methods.
- 304 - NOT MODIFIED - used in conditional GET requests to reduce the bandwidth use of the network. Here, the body of the response sent should be empty.
- 400 - BAD REQUEST - This can be due to validation errors or missing input data.
- 401 - FORBIDDEN - sent when the user does not have access (or is forbidden) to the resource.
- 404 - NOT FOUND - Resource method is not available.
- 500 - INTERNAL SERVER ERROR - server threw some exceptions while running the method.
- 502 - BAD GATEWAY - Server was not able to get the response from another upstream server.

8. What are the HTTP Methods?

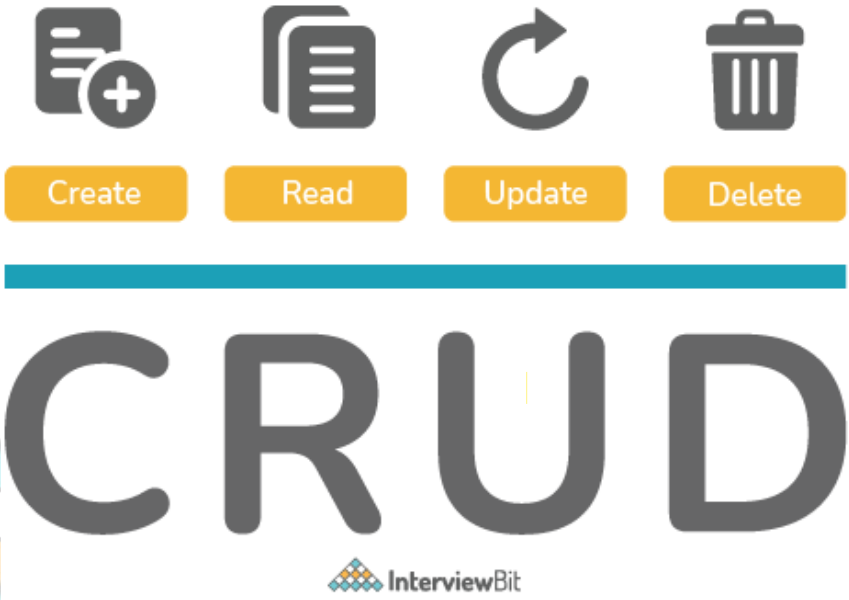
HTTP Methods are also known as HTTP Verbs. They form a major portion of uniform interface restriction followed by the REST that specifies what action has to be followed to get the requested resource. Below are some examples of HTTP Methods:

- GET: This is used for fetching details from the server and is basically a read-only operation.
- POST: This method is used for the creation of new resources on the server.
- PUT: This method is used to update the old/existing resource on the server or to replace the resource.
- DELETE: This method is used to delete the resource on the server.
- PATCH: This is used for modifying the resource on the server.
- OPTIONS: This fetches the list of supported options of resources present on the server.

OPTIONS method is used to determine the HTTP methods supported by a server for a specific

© Copyright by Interviewbit

The POST, GET, PUT, DELETE corresponds to the create, read, update, delete operations which are most commonly called **CRUD Operations**.



GET, HEAD, OPTIONS are safe and idempotent methods whereas PUT and DELETE methods are only idempotent. POST and PATCH methods are neither safe nor idempotent.

Safe methods → Don't change server state (read-only).
Idempotent methods → May change state,and if making one request has the same effect on the server as making the same r
multiple times.

9. Can you tell the disadvantages of RESTful web services?

The disadvantages are:

- As the services follow the idea of statelessness, it is not possible to maintain sessions. (Session simulation responsibility lies on the client-side to pass the session id)
- REST does not impose security restrictions inherently. It inherits the security measures of the protocols implementing it. Hence, care must be chosen to implement security measures like integrating SSL/TLS based authentications, etc.

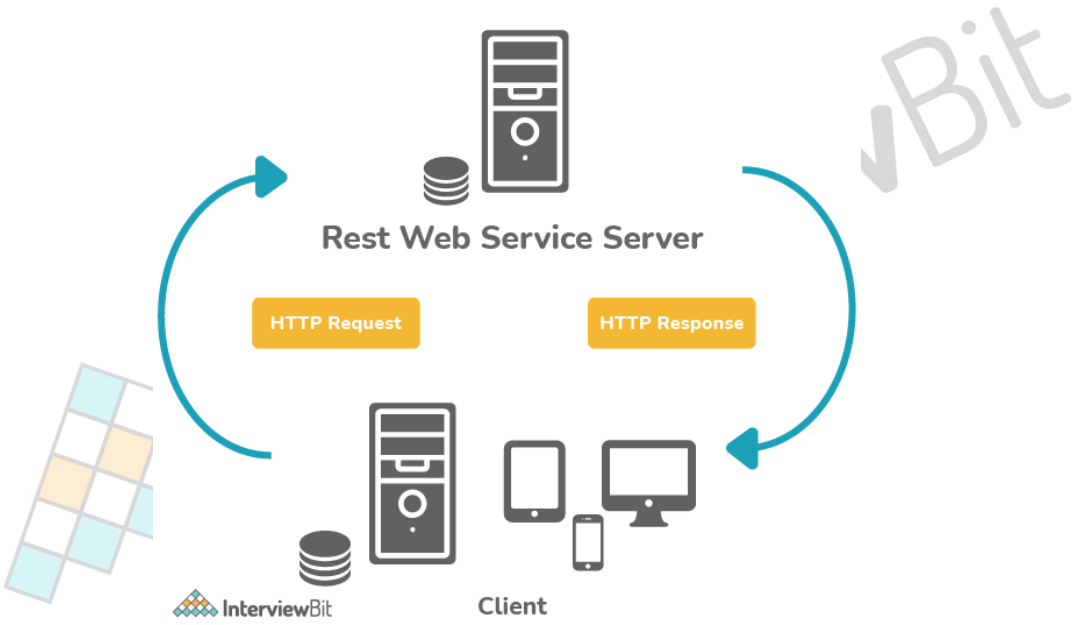
SSL (Secure Sockets Layer) → older protocol (now deprecated).
TLS (Transport Layer Security) → modern, secure replacement for S

TLS protocol used to encrypt communication between client and server.but JWT
Identifies the user/app making requests after the TLS and after the login ...read the "JWT" in nodes interview questions

10. Define Messaging in terms of RESTful web services.

© Copyright by Interviewbit

The technique of sending a message from the REST client to the REST server in the form of an HTTP request and the server responding back with the response as HTTP Response is called Messaging. The messages contained constitute the data and the metadata about the message.



REST API Experienced Interview Questions

11. Differentiate between SOAP and REST?

Web services are software components that allow different applications to communicate with each other over the internet, regardless of the programming language, platform, or

© Copyright by Interviewbit

SOAP	REST
SOAP - Simple Object Access Protocol	REST - Representational State Transfer
SOAP is a protocol used to implement web services.	REST is an architectural design pattern for developing web services
SOAP cannot use REST as it is a protocol.	REST architecture can have SOAP protocol as part of the implementation.
SOAP specifies standards that are meant to be followed strictly.	REST defines standards but they need not be strictly followed.
SOAP client is more tightly coupled to the server which is similar to desktop applications having strict contracts.	The REST client is more flexible like a browser and does not depend on how the server is developed unless it follows the protocols required for establishing communication.
SOAP supports only XML transmission between the client and the server.	REST supports data of multiple formats like XML, JSON, MIME, Text, etc.
SOAP reads are not cacheable.	REST read requests can be cached.
SOAP uses service interfaces for exposing the resource logic.	REST uses URI to expose the resource logic.

Web services are software protocol that allow different applications to communicate with each other over the internet, regardless of the programming language, platform, or operating system used.

© Copyright by Interviewbit

12. While creating URI for web services, what are the best practices that needs to be followed?

Below is the list of best practices that need to be considered with designing URI for web services:

- While defining resources, use plural nouns. Example: To identify user resource, use the name “users” for that resource.
- While using the long name for resources, use underscore or hyphen. Avoid using spaces between words. For example, to define authorized users resource, the name can be “authorized_users” or “authorized-users”.
- The URI is case-insensitive, but as part of best practice, it is recommended to use lower case only. Domain/host part: Not case sensitive... But Path, query, fragment: Case sensitive (depends on the server)
- While developing URI, the backward compatibility must be maintained once it gets published. When the URI is updated, the older URI must be redirected to the new one using the HTTP status code 300.
- Use appropriate HTTP methods like GET, PUT, DELETE, PATCH, etc. It is not needed or recommended to use these method names in the URI. Example: To get user details of a particular ID, use `/users/{id}` instead of `/getUser`
- Use the technique of forward slashing to indicate the hierarchy between the resources and the collections. Example: To get the address of the user of a particular id, we can use: `/users/{id}/address`

13. What are the best practices to develop RESTful web services?

RESTful web services use REST API as means of implementation using the HTTP protocol. REST API is nothing but an application programming interface that follows REST architectural constraints such as statelessness, cacheability, maintainability, and scalability. It has become very popular among the developer community due to its simplicity. Hence, it is very important to develop safe and secure REST APIs that follow good conventions. Below are some best practices for developing REST APIs:

© Copyright by Interviewbit

- Since REST supports multiple data formats, it is however good practice to develop REST APIs that accept and responds with JSON data format whenever possible. This is because a majority of the client and server technologies have inbuilt support to read and parse JSON objects with ease, thereby making JSON the standard object notation.
    - To ensure that the application responds using JSON data format, the response header should have Content-Type set to as `application/JSON` , this is because certain HTTP clients look at the value of this response header to parse the objects appropriately.
    - To ensure that the request sends the data in JSON format, again the Content-Type must be set to `application/JSON` on the request header.
- While naming the resource endpoints, ensure to use plural nouns and not verbs. The API endpoints should be clear, brief, easy to understand, and informative. Using verbs in the resource name doesn't contribute much information because an HTTP request already has what the request is doing in its HTTP method/verb. An appropriate HTTP verb should be used to represent the task of the API endpoint.
    - Below are the most commonly used HTTP methods to define the verb:
        - GET - indicates get/retrieve the resource data
        - POST - indicates create new resource data
        - PUT - indicates update the existing resource data
        - DELETE - indicates remove the resource data
- To represent the hierarchy of resources, use the nesting in the naming convention of the endpoints. In case, you want to retrieve data of one object residing in another object, the endpoint should reflect this to communicate what is happening. For example, to get the address of an author, we can use the GET method for the URI `/authors/:id/address'`
    - Please ensure there are no more than 2 or 3 levels of nesting as the name of the URI can become too long and unwieldy.
- Error Handling should be done gracefully by returning appropriate error codes the application has encountered. REST has defined standard HTTP Status codes that can be sent along with the response based on the scenario.
    - Error codes should also be accompanied by appropriate error messages that can help the developers to take corrective actions. However, the message should not be too elaborate as well which can help the hacker to hack your application.

© Copyright by Interviewbit

14. What are Idempotent methods? How is it relevant in RESTful web services domain?

The meaning of idempotent is that even after calling a single request multiple times, the outcome of the request should be the same. While designing REST APIs, we need to keep in mind to develop idempotent APIs. This is because the consumers can write client-side code which can result in duplicate requests intentionally or not. Hence, fault-tolerant APIs need to be designed so that they do not result in erroneous responses.

© Copyright by Interviewbit

- Idempotent methods ensure that the responses to a request if called once or ten times or more than that remain the same. This is equivalent to adding any number with 0.
- REST provides idempotent methods automatically. GET, PUT, DELETE, HEAD, OPTIONS, and TRACE are the idempotent HTTP methods. POST is not idempotent.
    - POST is not idempotent because POST APIs are usually used for creating a new resource on the server. While calling POST methods N times, there will be N new resources. This does not result in the same outcome at a time.
    - Methods like GET, OPTIONS, TRACE, and HEAD are idempotent because they do not change the state of resources on the server. They are meant for resource retrieval whenever called. They do not result in write operations on the server thereby making it idempotent.
    - PUT methods are generally used for updating the state of resources. If you call PUT methods N times, the first request updates the resource and the subsequent requests will be overwriting the same resource again and again without changing anything. Hence, PUT methods are idempotent.
    - DELETE methods are said to be idempotent because when calling them for N times, the first request results in successful deletion (Status Code 200), and the next subsequent requests result in nothing - Status Code 204. The response is different, but there is no change of resources on the server-side.
        - However, if you are attempting to delete the resource present, at last, every time you hit the API, such as the request `DELETE /user/last` which deletes the last user record, then calling the request N times would delete N resources on the server. This does not make DELETE idempotent. In such cases, as part of good practices, it is advisable to use POST requests.

15. What are the differences between REST and AJAX?

© Copyright by Interviewbit

REST	AJAX
REST- Representational State Transfer	AJAX - Asynchronous javascript and XML
REST has a URI for accessing resources by means of a request-response pattern.	AJAX uses XMLHttpRequest object to send requests to the server and the response is interpreted by the Javascript code dynamically.
REST is an architectural pattern for developing client-server communication systems.	AJAX is used for dynamic updation of UI without the need to reload the page.
REST requires the interaction between client and server.	AJAX supports asynchronous requests thereby eliminating the necessity of constant client-server interaction.

XMLHttpRequest (XHR) is a JavaScript built-in object used to send HTTP requests from a web browser to a server and receive data without reloading the page.

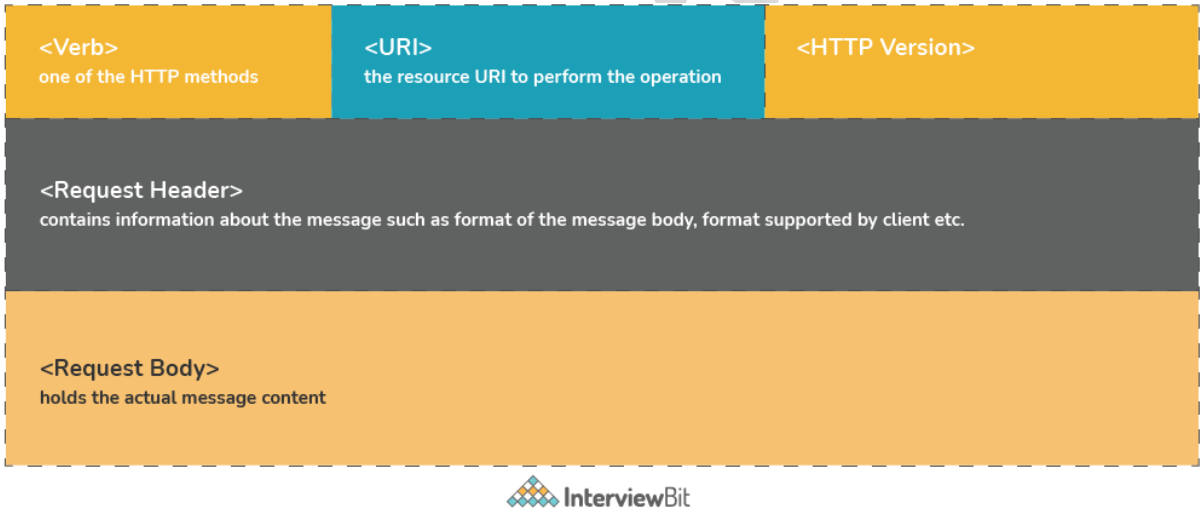
REST API live and work in backend side but AJAX used to send data form frontend to backend but now we use somethong else .....

16. Can you tell what constitutes the core components of HTTP Request?

In REST, any HTTP Request has 5 main components, they are:

© Copyright by Interviewbit

- Method/Verb – This part tells what methods the request operation represents. Methods like GET, PUT, POST, DELETE, etc are some examples.
- URI – This part is used for uniquely identifying the resources on the server.
- HTTP Version – This part indicates what version of HTTP protocol you are using. An example can be HTTP v1.1.
- Request Header – This part has the details of the request metadata such as client type, the content format supported, message format, cache settings, etc.
- Request Body – This part represents the actual message content to be sent to the server.

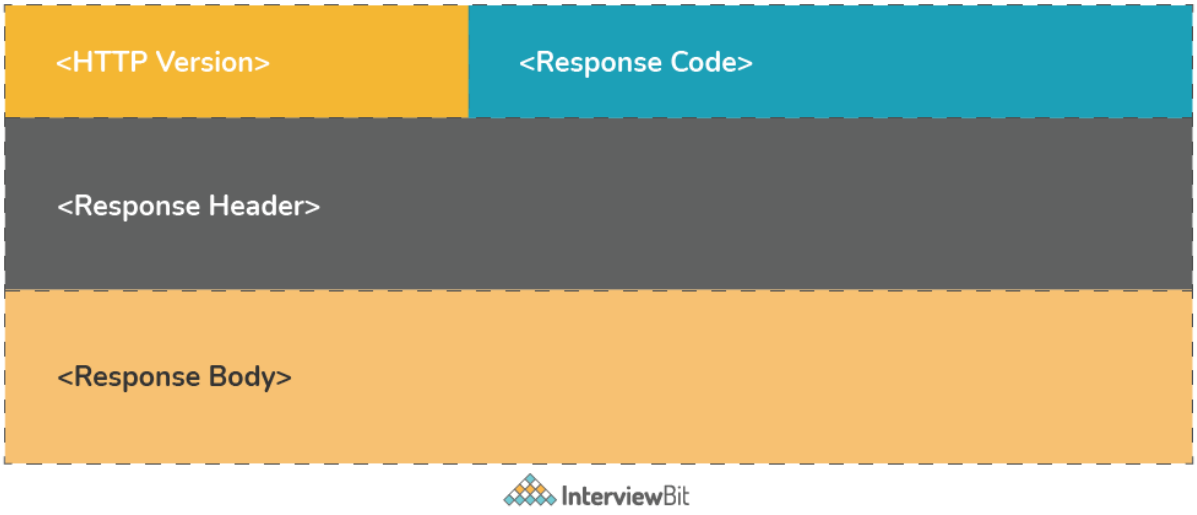


17. What constitutes the core components of HTTP Response?

HTTP Response has 4 components:

© Copyright by Interviewbit

- Response Status Code – This represents the server response status code for the requested resource. Example- 400 represents a client-side error, 200 represents a successful response.
- HTTP Version – Indicates the HTTP protocol version.
- Response Header – This part has the metadata of the response message. Data can describe what is the content length, content type, response date, what is server type, etc.
- Response Body – This part contains what is the actual resource/message returned from the server.



18. Define Addressing in terms of RESTful Web Services.

Addressing is the process of locating a single/multiple resources that are present on the server. This task is accomplished by making use of URI (Uniform Resource Identifier). The general format of URI is

```
<protocol>://<application-name>/<type-of-resource>/<id-of-resource>
```

19. What are the differences between PUT and POST in REST?

© Copyright by Interviewbit

PUT	POST
PUT methods are used to request the server to store the enclosed entity in request. In case, the request does not exist, then new resource has to be created. If the resource exists, then the resource should get updated.	POST method is used to request the server to store the enclosed entity in the request as a new resource.
The URI should have a resource identifier. Example: `PUT /users/{user-id}`	The POST URI should indicate the collection of the resource. Example: `POST /users`
PUT methods are idempotent.	POST methods are not idempotent.
PUT is used when the client wants to modify a single resource that is part of the collection. If a part of the resource has to be updated, then PATCH needs to be used.	POST methods are used to add a new resource to the collection.
The responses are not cached here despite the idempotency.	Responses are not cacheable unless the response explicitly specifies Cache-Control fields in the header.

© Copyright by Interviewbit

20. What makes REST services to be easily scalable?

REST services follow the concept of statelessness which essentially means no storing of any data across the requests on the server. This makes it easier to scale horizontally because the servers need not communicate much with each other while serving requests.

21. Based on what factors, you can decide which type of web services you need to use - SOAP or REST?

REST services have gained popularity due to the nature of simplicity, scalability, faster speed, improved performance, and multiple data format support. But, SOAP has its own advantages too. Developers use SOAP where the services require advanced security and reliability.

Following are the questions you need to ask to help you decide which service can be used:

© Copyright by Interviewbit

- Do you want to expose resource data or business logic?
  - SOAP is commonly used for exposing business logic and REST for exposing data.
- Does the client require a formal strict contract?
  - If yes, SOAP provides strict contracts by using WSDL. Hence, SOAP is preferred here.
- Does your service require support for multiple formats of data?
  - If yes, REST supports multiple data formats which is why it is preferred in this case.
- Does your service require AJAX call support?
  - If yes, REST can be used as it provides the XMLHttpRequest.
- Does your service require both synchronous and asynchronous requests?
  - SOAP has support for both sync/async operations.
  - REST only supports synchronous calls.
- Does your service require statelessness?
  - If yes, REST is suitable. If no, SOAP is preferred.
- Does your service require a high-security level?
  - If yes, SOAP is preferred. REST inherits the security property based on the underlying implementation of the protocol. Hence, it can't be preferred at all times.
- Does your service require support for transactions?
  - If yes, SOAP is preferred as it is good in providing advanced support for transaction management.
- What is the bandwidth/resource required?
  - SOAP involves a lot of overhead while sending and receiving XML data, hence it consumes a lot of bandwidth.
  - REST makes use of less bandwidth for data transmission.
- Do you want services that are easy to develop, test, and maintain frequently?
  - REST is known for simplicity, hence it is preferred.

© Copyright by Interviewbit

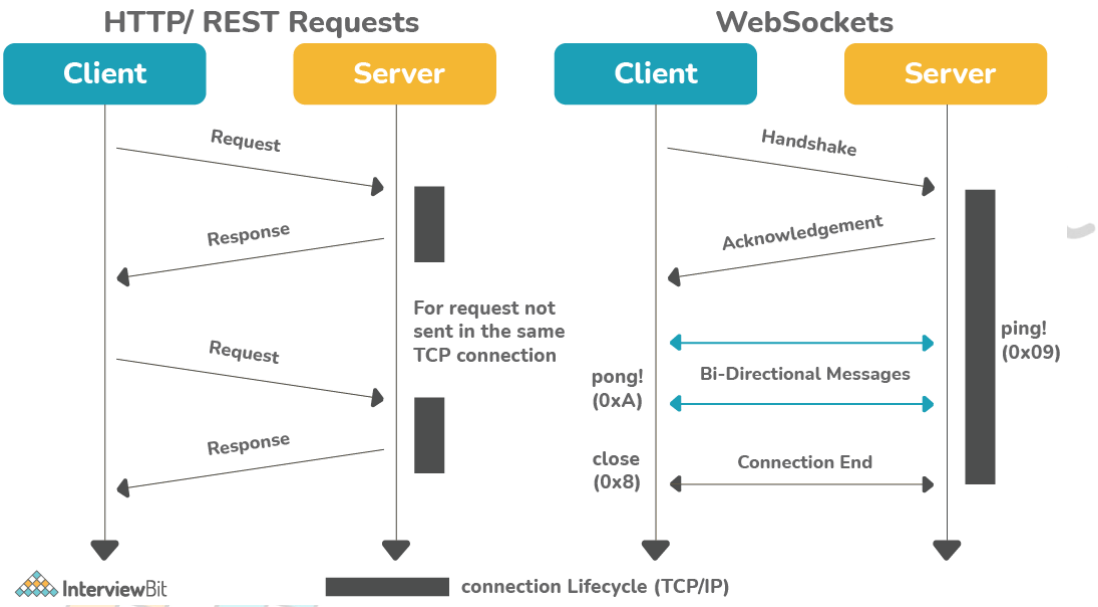
22. We can develop webservices using web sockets as well as REST. What are the differences between these two?

© Copyright by Interviewbit

REST	Web Socket
REST follows stateless architecture, meaning it won't store any session-based data.	Web Socket APIs follow the stateful protocol as it necessitates session-based data storage.
The mode of communication is uni-directional. At a time, only the server or the client will communicate.	The communication is bi-directional, communication can be done by both client or server at a time.
REST is based on the Request-Response Model.	Web Socket follows the full-duplex model.
Every request will have sections like header, title, body, URL, etc.	Web sockets do not have any overhead and hence suited for real-time communication.
For every HTTP request, a new TCP connection is set up.	There will be only one TCP connection and then the client and server can start communicating.
REST web services support both vertical and horizontal scaling.	Web socket-based services only support vertical scaling.
REST depends on HTTP methods to get the response.	Web Sockets depend on the IP address and port number of the system to get a response.
Communication is slower here.	Message transmission happens very faster than REST API

© Copyright by Interviewbit

The request flow difference between the REST and Web Socket is shown below:



23. Can we implement transport layer security (TLS) in REST?

Yes, we can. TLS does the task of encrypting the communication between the REST client and the server and provides the means to authenticate the server to the client. It is used for secure communication as it is the successor of the Secure Socket Layer (SSL). HTTPS works well with both TLS and SSL thereby making it effective while implementing RESTful web services. One point to mention here is, the REST inherits the property of the protocol it implements. So security measures are dependent on the protocol REST implements.

24. Should we make the resources thread safe explicitly if they are made to share across multiple clients?

There is no need to explicitly making the resources thread-safe because, upon every request, new resource instances are created which makes them thread-safe by default. Question want to say that-If many users send requests to the same API at the same time, do we need to manually handle thread safety (synchronization, locks, etc.) in the resource class?
ans: In most REST frameworks (like JAX-RS), a new resource object is created for every request. Since each request gets its own instance, multiple threads are not sharing the same object.

25. What is Payload in terms of RESTful web services?

© Copyright by Interviewbit

Payload refers to the data passes in the request body. It is not the same as the request parameters. The payload can be sent only in POST methods as part of the request body.

26. Is it possible to send payload in the GET and DELETE methods?

No, the payload is not the same as the request parameters. Hence, it is not possible to send payload data in these methods.

27. How can you test RESTful Web Services?

RESTful web services can be tested using various tools like Postman, Swagger, etc. Postman provides a lot of features like sending requests to endpoints and show the response which can be converted to JSON or XML and also provides features to inspect request parameters like headers, query parameters, and also the response headers. Swagger also provides similar features like Postman and it provides the facility of documentation of the endpoints too. We can also use tools like Jmeter for performance and load testing of APIs.

28. What is the maximum payload size that can be sent in POST methods?

Theoretically, there is no restriction on the size of the payload that can be sent. But one must remember that the greater the size of the payload, the larger would be the bandwidth consumption and time taken to process the request that can impact the server performance.

29. How does HTTP Basic Authentication work?

While implementing Basic Authentication as part of APIs, the user must provide the username and password which is then concatenated by the browser in the form of “username: password” and then perform base64 encoding on it. The encoded value is then sent as the value for the “Authorization” header on every HTTP request from the browser. Since the credentials are only encoded, it is advised to use this form when requests are sent over HTTPS as they are not secure and can be intercepted by anyone if secure protocols are not used.

Read in details: https://docs.google.com/document/d/1bMqrN38SZYRTv-r-nUAZglrZBhZB-yPhd3GPNOHOEYQ/edit?

© Copyright by Interviewbit

30. What is the difference between idempotent and safe HTTP methods?

- Safe methods are those that do not change any resources internally. These methods can be cached and can be retrieved without any effects on the resource.
- Idempotent methods are those methods that do not change the responses to the resources externally. They can be called multiple times without any change in the responses.

According to restcookbook.com, the following is the table that describes what methods are idempotent and what is safe.

HTTP Methods	Idempotent	Safe
OPTIONS	yes	yes
GET	yes	yes
HEAD	yes	yes
PUT	yes	no
POST	no	no
DELETE	yes	no
PATCH	no	no

JAX-RS Interview Questions

31. What are the key features provided by JAX-RS API in Java EE?

© Copyright by Interviewbit

A RESTful Web Service is an API that:

.Exposes resources (data or services) via URIs (Uniform Resource Identifiers).

.Uses HTTP methods to perform operations on those resources:

GET – Retrieve data

POST – Create data

PUT – Update data

DELETE – Delete data

.Is stateless – each request contains all necessary information; server doesn’t store client context.

.Returns data in formats like JSON or XML.

.Uses HTTP status codes to indicate operation results (e.g., 200 OK, 404 Not Found, 201 Created).